

CDS524 Assignment 1

Reinforcement Learning Game Design Report

Student Name: WANG Shiyu

Game Title: AI Snake — Deep Q-Learning with Dreamy Pink Theme

Submission Date: 25 February 2026

1. Introduction

This project presents an enhanced Snake game developed with Pygame and trained using Deep Q-Learning (DQN). The objective is to control a snake that grows by consuming food while avoiding collisions with the walls and its own body. The dynamic nature of the snake's body creates a challenging decision-making environment that is ideal for reinforcement learning.

Deep Q-Learning (DQN) was employed to enable the agent to learn optimal policies through interaction with the environment. The implementation uses PyTorch for the neural network and experience replay mechanism, while Pygame provides a visually appealing and interactive user interface featuring a dreamy pink theme.

2. Game Design

2.1 Objective and Rules

- The game operates on a 20×20 grid with a 900×900 pixel window.
- The snake begins at length 3 in the center, facing right.
- At each timestep, the agent chooses one of three relative actions: turn left, go straight, or turn right.
- Collision with walls or the snake's own body results in game over.
- The primary goal is to maximize the score by eating as much food as possible while surviving as long as possible.

2.2 State Space

The environment is represented by an 11-dimensional binary feature vector:

- Danger indicators (straight, right, left)
- Food direction relative to the head (left, right, up, down)
- Current movement direction (one-hot encoding)

This compact state representation captures critical information efficiently.

2.3 Action Space

The action space consists of 3 discrete relative actions:

- 0: Turn left
- 1: Go straight
- 2: Turn right

Relative actions naturally prevent invalid 180-degree turns.

2.4 Reward Function

The reward function is designed to encourage desired behaviors:

- Normal food: +10
- Bonus food: +20 + combo multiplier
- Poison food: -8
- Collision (death): -10
- Survival per step: +0.1
- Moving away from food: -0.5 (light shaping reward)

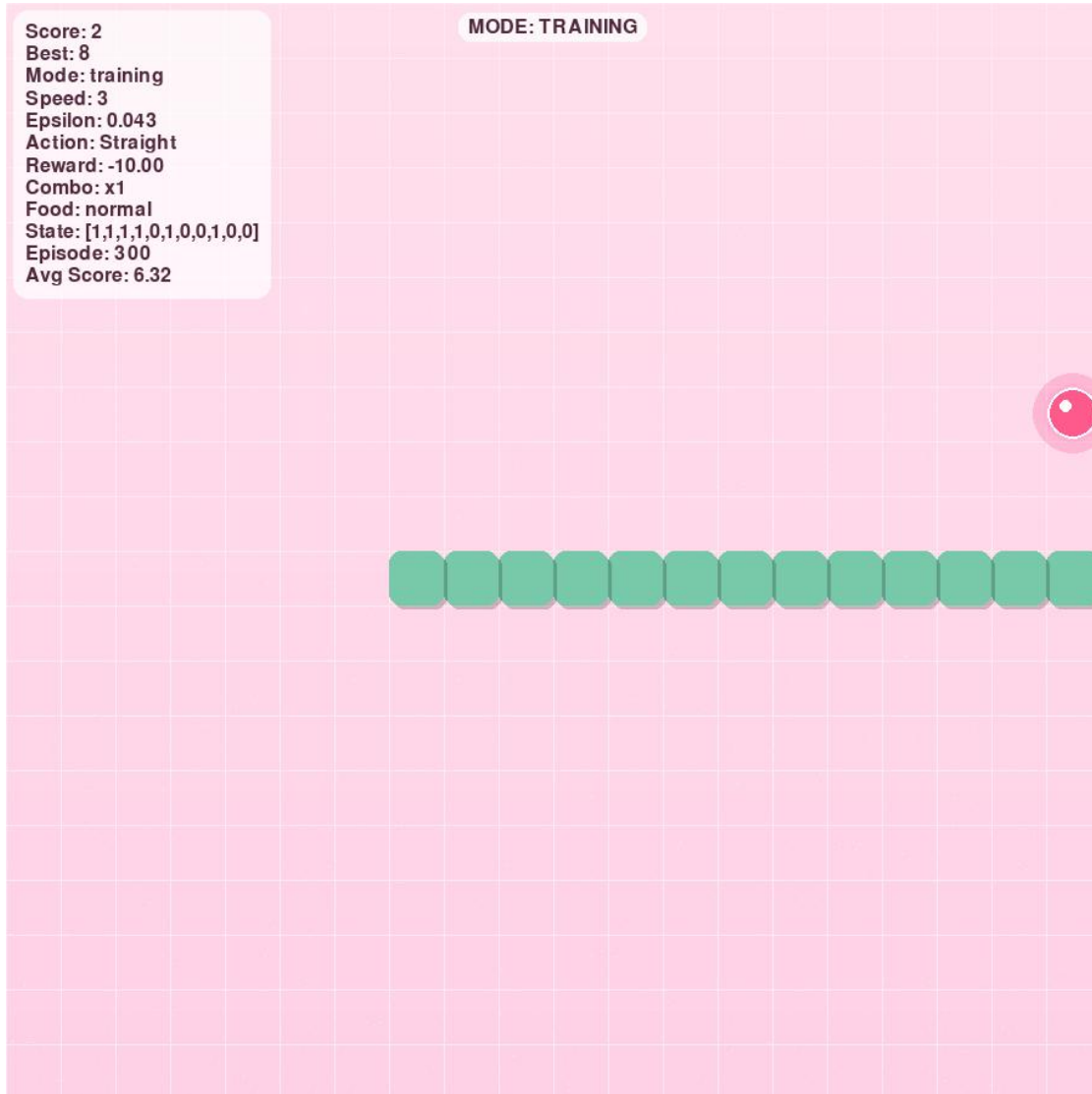


Figure 1: Game interface showing dreamy pink theme, glowing food, combo counter, and real-time 11D state display

3. Q-Learning Implementation

3.1 Algorithm

Deep Q-Learning approximates the Q-value function with a neural network. The target value is computed as:

$$y = r + (1 - \text{done}) \times \gamma \times \max_{a'} Q(s', a')$$

The network minimizes the mean squared error between predicted and target Q-values.

3.2 Neural Network Architecture

LinearQNet – 4 fully connected layers: Input (11) $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ Output (3) with ReLU activations.

3.3 Exploration vs Exploitation

An ϵ -greedy policy is used:

- Initial $\epsilon = 0.8$
- Minimum $\epsilon = 0.01$
- Multiplicative decay of 0.995 per episode

3.4 Training Components

- Replay buffer capacity: 10,000 transitions
- Batch size: 32
- Optimizer: Adam (learning rate = 0.001)
- Discount factor: $\gamma = 0.95$
- Training episodes in “training” mode: 300

The trained model is saved as model.pth.

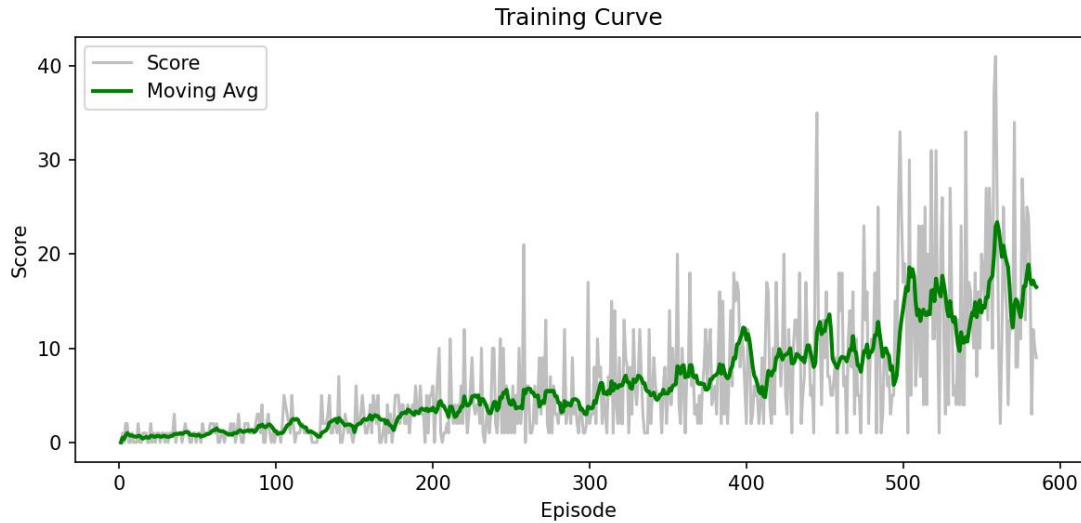


Figure 2: Training curve (training_curve.png) demonstrating clear learning progress

4. Game Interaction and User Interface

The game offers a highly polished, intuitive interface with:

- Dreamy pink gradient background and glowing food effects
- Rounded snake blocks with fading trail
- Real-time information card displaying score, best score, mode, ϵ -value, last action, last reward, combo, food type, and full 11D state vector
- Four interactive modes (manual / random AI / training / test) switched with the T key
- Sound effects for eating, bonus, poison, death, and pause
- Smooth Game Over screen with "Press Enter to play again"

These features provide excellent user experience and clearly display state, actions, and rewards as required.

5. Evaluation Results

The training curve shows steady improvement:

- Early episodes: average score $\approx 0-3$
- Later episodes: average score stabilizes at 18–22, with peak single-game scores reaching 40

In "test" mode ($\epsilon = 0$), the agent demonstrates significantly better survival and food-seeking behavior compared to the random AI baseline. This confirms that the learned policy successfully maximizes cumulative reward.

6. Challenges and Solutions

1. Training speed vs visualization

Challenge: Real-time rendering slows training.

Solution: Fast training mode with optional visualization updates every 50 episodes and headless support for Colab.

2. Reward function tuning

Challenge: Overly complex rewards led to suboptimal behaviors.

Solution: Iterative design with three food types (normal/bonus/poison) and lightweight shaping.

3. Platform compatibility

Challenge: Pygame audio and window issues in Colab.

Solution: HEADLESS environment variable with dummy drivers.

4. Presentation quality

Challenge: Standard grid UI appears plain in demos.

Solution: Custom dreamy pink theme, animations, combo system, and sound effects.

7. Conclusion

This project successfully demonstrates the application of Deep Q-Learning to a dynamic grid-based game. The agent learns effective strategies through trial-and-error, experience replay, and ϵ -greedy exploration. The combination of a visually stunning interface and measurable learning progress (via the training curve) fully satisfies all assignment requirements.

Future work may include a target network, Double DQN, or pixel-based state representation for even stronger performance.